

PROGRAMMAZIONE II

DOCUMENTAZIONE E UNIT TESTING

Michele Pasqua, PhD



UNIVERSITÀ
di **VERONA**
Dipartimento
di **INFORMATICA**

A.A. 2025/2026

DOCUMENTAZIONE



DOCUMENTAZIONE AUTOMATIZZATA

Documentare il codice ne migliora la comprensione e la manutenibilità

- ▶ Per te e i tuoi collaboratori
- ▶ Per chiunque voglia usare il tuo codice

La documentazione può essere **generata automaticamente** usando javadoc

- ▶ Il programmatore scrive la documentazione come commenti al codice
- ▶ Il tool javadoc genera automaticamente pagine web
- ▶ Scrivere la documentazione più velocemente e in modo standard

DOCUMENTAZIONE AUTOMATIZZATA (CONT.)

Il tool `javadoc` comprende solo i **doc comment**

- ▶ Sono racchiusi tra `/**` e `*/`
- ▶ Sono permesse più righe, che iniziano con `*`
- ▶ Possono contenere tag HTML

```
/**  
 * Questo è un doc comment con cui alimentare javadoc  
 */
```

1
2
3

TAG DI DOCUMENTAZIONE

Il tool javadoc supporta dei **tag** per analizzare i doc comment

- ▶ Iniziano con @ e sono case-sensitive
- ▶ Devono trovarsi all'inizio della riga

```
/** 1
 * Stampa il k-esimo elemento di un array di String 2
 * 3
 * @author Tony Stark 4
 * @param arr L'array il cui elemento deve essere stampato 5
 * @param k L'indice dell'elemento da stampare 6
 */ 7
public void printK(String[] arr, int k) { System.out.println(arr[k]); } 8
```

TAG DI DOCUMENTAZIONE (CONT.)

Tag importanti

- ▶ `@param`: descrive uno specifico parametro di un metodo
- ▶ `@return`: descrive il valore di ritorno di un metodo
- ▶ `@throws`: descrive le eccezioni che un metodo può lanciare
- ▶ `@link`: fornisce un puntatore a una classe
- ▶ `@deprecated`: fornisce una spiegazione del perché il codice è deprecato

DOCUMENTARE UNA CLASSE

Per documentare una classe basta inserire un doc comment sopra di essa

```
/** 1
 * Una classe che modella studenti universitari 2
 * 3
 * È una sottoclasse della classe {@link pkg.Person} 4
 * @author Bruce Banner 5
 */ 6
public class Student extends Person { 7
    // campi e metodi 8
} 9
```

DOCUMENTARE UN METODO

Per documentare un metodo basta inserire un doc comment sopra di esso

```
/** 1
 * <p>Un metodo che calcola il quadrato di un numero 2
 * <a href="https://en.wikipedia.org/wiki/Square_(algebra)">Vedi altro</a> 3
 * </p> 4
 * 5
 * @param num Il numero da elevare al quadrato 6
 * @return Il quadrato del numero fornito 7
 * @since 1.0 8
 */ 9
public int computePow(int num) { 10
    return pow * pow; 11
} 12
```

DOCUMENTARE LE VARIABILI

Per documentare una variabile basta inserire un doc comment sopra di essa

```
/**  
 * Il nome pubblico di una persona  
 */  
public String name;  
  
/**  
 * L'età privata di una persona  
 */  
private int age;
```

1
2
3
4
5
6
7
8
9

I campi privati non vengono documentati, a meno che l'opzione `-private` non sia passata a javadoc

GENERARE LA DOCUMENTAZIONE

Tramite la riga di comando

```
javadoc -d output packages/classes
```

Per generare la documentazione per tutti i package e le classi nella directory corrente

```
javadoc -private -d doc *
```

Le pagine web sono salvate nella directory `doc`, considerando campi e metodi privati

- È necessario aggiungere flag per alcuni tag, come `@author` e `@since`

GENERARE LA DOCUMENTAZIONE (CONT.)

Utilizzando un tool di build come Maven con un plugin adatto

```
<build>                                <!-- da aggiungere al file di configurazione Maven pom.xml -->
  <plugins>
    <plugin>
      <groupId>org.apache.maven.plugins</groupId>
      <artifactId>maven-javadoc-plugin</artifactId>
      <version>3.6.2</version>
      <configuration>
        <source>1.8</source>
        <show>private</show>
      </configuration>
    </plugin>
  </plugins>
</build>
```

Eseguire con `mvn javadoc:javadoc`

UNIT TESTING



TESTING

Modalità programmatica di verifica della **qualità del software**

- ▶ Verifica che il codice si comporti come previsto
- ▶ Aiuta durante lo sviluppo del codice
- ▶ Garantisce coerenza durante l'evoluzione del codice
- ▶ Migliora la comprensione del codice
- ▶ Riduce il tempo di debugging

// sviluppo guidato dai test (TDD)

// test di regressione

Il testing può essere **manuale** o automatizzato

- ▶ Il testing automatizzato può essere usato per individuare bug o vulnerabilità

TESTING (CONT.)

Un caso di test rappresenta uno scenario per l'esecuzione del codice, comprendente

- ▶ Un **input** per il codice *// valori, azioni, eventi*
- ▶ Un **output atteso** per il codice

Un caso di test ha successo se l'output atteso viene prodotto in risposta a un input

- ▶ Tipicamente, vengono testati componenti isolati (es. una classe)

UNIT TESTING IN JAVA

JUnit è un framework per scrivere **unit test** di codice Java

- ▶ Implementato all'interno di Java, usando classi e annotazioni specifiche

Testare una singola classe Java

- ▶ Un caso di test testa un singolo metodo della classe
- ▶ Una test suite è una collezione di casi di test

UNIT TESTING IN JAVA (CONT.)

JUnit esegue una test suite e riporta i risultati

Per ogni caso di test nella test suite

- ▶ JUnit chiama il metodo dai casi di test
- ▶ Se il metodo termina senza errori, il caso di test è superato

Tecnicamente, un caso di test è un metodo senza valore di ritorno

- ▶ Se il test fallisce, il metodo lancia un `AssertionFailedError`
- ▶ Il framework JUnit cattura l'errore e lo gestisce in modo trasparente

ASSERZIONI

Il package `org.junit.jupiter.api` contiene la classe `Assertions`

// JUnit 5

La classe fornisce metodi per specificare asserzioni

```
static void assertTrue(condition, message)
```

Se *condition* viene valutata a

- ▶ `true`: l'esecuzione continua all'istruzione successiva
- ▶ `false`: l'esecuzione salta alla fine del metodo (e *message* viene visualizzato)
- ▶ Il messaggio di errore è opzionale

ASSERTZIONI (CONT.)

Il package `org.junit.jupiter.api` contiene la classe `Assertions`

// JUnit 5

La classe fornisce metodi per specificare asserzioni

```
static void assertEquals(expected, actual, message)
```

Se *expected* è uguale ad *actual*

// funziona per valori primitivi e oggetti

- ▶ Allora l'esecuzione continua all'istruzione successiva
- ▶ Altrimenti l'esecuzione salta alla fine del metodo (e *message* viene visualizzato)
- ▶ Il messaggio di errore è opzionale

ASSERZIONI (CONT.)

Il package `org.junit.jupiter.api` contiene la classe `Assertions`

// JUnit 5

La classe fornisce metodi per specificare asserzioni

```
static void assertEquals(expected, actual, message)
```

Se *expected* è uguale ad *actual*

// funziona per array primitivi e di oggetti

- ▶ Allora l'esecuzione continua all'istruzione successiva
- ▶ Altrimenti l'esecuzione salta alla fine del metodo (e *message* viene visualizzato)
- ▶ Il messaggio di errore è opzionale

PRIMO UNIT TEST

```
import org.junit.jupiter.api.Test;           1
import org.junit.jupiter.api.BeforeAll;      2
import org.junit.jupiter.api.Assertions;      3

public class StudentTest { // aggiungi 'Test' 4

    private static Student student; // oggetto della classe da testare 5

    @BeforeAll // eseguito per primo           6
    void static setup() { student = new Student("Sam", 123); } 7

    @Test                                         8
    public void testGetMatricola() { // anteponi 'test' 9
        int actual = student.getMatricola();    10
        Assertions.assertEquals(123, actual, "Exp: 123, Act: " + actual); 11
    }                                             12
}                                                13
}                                                14
}                                                15
}                                                16
}                                                17
```

PRIMO UNIT TEST (CONT.)

Per semplificare il codice, è possibile usare `import static`

Quando una classe fornisce un metodo `static`

- ▶ Importare solo quel metodo, evitando il nome qualificato completo quando lo si usa

```
import static org.junit.jupiter.api.Assertions.assertEquals;

...
@Test
public void testGetMatricola() {
    int actual = student.getMatricola();
    assertEquals(123, actual, "Exp: 123, Act: " + actual);
}
```

- 1
- 2
- 3
- 4
- 5
- 6
- 7
- 8

ANNOTAZIONI JUNIT

@DisplayName	Nome personalizzato per una classe o un metodo di test
@BeforeEach	Il metodo annotato verrà eseguito prima di ogni metodo di test
@AfterEach	Il metodo annotato verrà eseguito dopo ogni metodo di test
@BeforeAll	Il metodo annotato verrà eseguito prima di tutti i metodi di test
@AfterAll	Il metodo annotato verrà eseguito dopo tutti i metodi di test
@Disabled	Disabilita una classe o un metodo di test

ORGANIZZAZIONE DEL PROGETTO

Alle classi di test viene tipicamente aggiunto in coda `Test`

- ▶ Tipicamente si usa una cartella differente per le classi di test

Convenzione di denominazione di Maven e Gradle

- ▶ `src/main/java` contiene le classi sorgente del progetto
- ▶ `src/test/java` contiene le classi di test del progetto

I test dovrebbero essere coerenti con la struttura dei package

- ▶ Una classe di test `some.pkg.ClazzTest`
- ▶ Dovrebbe corrispondere a una classe sorgente `some.pkg.Clazz`

ESECUZIONE DEI TEST

Utilizzando un tool di build come Maven con un plugin adatto

```
<dependencies>          <!-- da aggiungere al file di configurazione Maven pom.xml -->
  <dependency>
    <groupId>org.junit.jupiter</groupId>
    <artifactId>junit-jupiter-engine</artifactId>
    <version>5.10.0</version>
    <scope>test</scope>
  </dependency>
  <dependency>
    <groupId>org.apache.maven.plugins</groupId>
    <artifactId>maven-surefire-plugin</artifactId>
    <version>3.5.4</version>
  </dependency>
</dependencies>
```

Eseguire con `mvn test`



Q + A

